

Entregable 1: Datos:

Aarón Blanco Álvarez

Máster en ingeniería informática

**Universidad de Granada
Facultad de Ingeniería Informática**



**UNIVERSIDAD
DE GRANADA**

Documentación del Modelo de Datos - MiauMarket

1. Introducción

La interfaz del repository define como se hace el acceso a datos, y la implementación conecta con los datos: backend con Retrofit y sesión local con DataStore.

En auth, el repository guarda y borra el token JWT. En products, el repository consulta el catálogo, aplica paginación y obtiene el detalle de cada producto.

La UI nunca accede directamente a Retrofit ni a DataStore; siempre pasa por el repository.

2. Persistencia local en Android (DataStore)

Nombre: *session_prefs*

Representa la sesión activa en el dispositivo.

Campo	Tipo Kotlin	Obligatorio	Descripción
jwt_token	String	No	JWT para enviar en <code>Authorization: Bearer <token></code> .

Ejemplo real:

JSON

```
{
  "jwt_token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxZW1haWwiOiJwZXBIQGV4YW1wbG
  UuY29tliwicz9sZSI6InVzZXlifQ.signature"
}
```

3. APIs que consume Android

Las consultas a el backend cuya ruta es */api*.

3.1. Auth

POST */api/auth/register*

Escritura de usuario y apertura de sesión.

Request:

Campo	Obligatorio	Tipo Kotlin
firstName	Sí	String
lastName	Sí	String
birthDate	Sí	String (yyyy-MM-dd)
email	Sí	String
password	Sí	String
role	No (default user)	String

Response (201):

JSON

```
{
  "token": "<jwt>",

```

```

"user": {
  "id": 1,
  "firstName": "Pepe",
  "lastName": "Garcia",
  "birthDate": "1998-05-21T00:00:00.000Z",
  "email": "pepe@example.com",
  "role": "user"
}
}

```

POST /api/auth/login

Inicio de sesión.

Request:

Campo	Obligatorio	Tipo Kotlin
email	Sí	String
password	Sí	String

Response (200):

JSON

```

{
  "token": "<jwt>",
  "user": {
    "id": 1,
    "firstName": "Pepe",
    "lastName": "Garcia",
    "birthDate": "1998-05-21T00:00:00.000Z",
    "email": "pepe@example.com",
    "role": "user"
  }
}

```

GET /api/auth/me

Lectura del usuario autenticado.

Header recomendado en Android: Authorization: Bearer <jwt_token>

Response (200):

JSON

```

{
  "user": {
    "id": 1,
    "firstName": "Pepe",
    "lastName": "Garcia",
    "birthDate": "1998-05-21T00:00:00.000Z",
    "email": "pepe@example.com",
    "role": "user"
  }
}

```

POST /api/auth/logout

Cierre de sesión.

Response (200):

JSON

```

{ "ok": true }

```

3.2. Products

GET /api/products

Catálogo con la parte de páginas.

Request:

Campo	Tipo	Obligatorio
search	String	No
take	Int	No
skip	Int	No

Response (200):

JSON

```
{
  "total": 128,
  "take": 20,
  "skip": 0,
  "items": [
    {
      "id": 57,
      "source": "kiwoko",
      "title": "Rascador arbol para gatos",
      "price": 45.95,
      "rawPrice": "45.95 EUR",
      "currency": "EUR",
      "url": "https://www.kiwoko.com/gatos/rascador-x",
      "image": "https://cdn.kiwoko.com/rascador-x.jpg",
      "scrapedAt": "2026-04-07T10:32:12.000Z",
      "createdAt": "2026-04-07T10:33:01.000Z",
      "updatedAt": "2026-04-07T10:33:01.000Z"
    }
  ]
}
```

Otros Endpoints de Operaciones

- **GET** /api/products/:id: Lectura de detalle por id numérico.
- **POST** /api/products/:id: Creación manual de producto (requiere rol admin).
- **PUT** /api/products/:id: Actualización de producto (requiere rol admin).
- **DELETE** /api/products/:id: Borrado de producto (requiere rol admin).

4. DTOs Kotlin

ProductResponse

Campo	Tipo Kotlin	Obligatorio
id	Long	Sí
source	String	Sí
title	String	Sí
price	Double?	No
rawPrice	String?	No
currency	String	Sí
url	String	Sí
image	String?	No

scrapedAt	String	Sí
createdAt	String	Sí
updatedAt	String	Sí

ProductsListResponse

Campo	Tipo Kotlin	Obligatorio
total	Int	Sí
take	Int	Sí
skip	Int	Sí
items	List<ProductResponse>	Sí

UserResponse

Campo	Tipo Kotlin	Obligatorio
id	Long	Sí
firstName	String	Sí
lastName	String	Sí
birthDate	String	Sí
email	String	Sí
role	String	Sí

AuthResponse

Campo	Tipo Kotlin	Obligatorio
token	String	Sí
user	UserResponse	Sí

4. Relaciones y cardinalidad relevantes para Android

La app no usa Room ni una base de datos relacional local, sino que consume un backend REST y guarda únicamente la sesión en DataStore. Por eso, las relaciones entre datos son lógicas y vienen definidas por la API:

- **Usuario: Sesión en dispositivo (1 : 0..1):** Un usuario puede tener como máximo un *jwt_token* guardado en el dispositivo. Esa sesión se persiste en DataStore dentro de *session_prefs*.
- **Products (lista): Product (detalle) (1 : N):** El endpoint */api/products* devuelve varios productos dentro de *items*, y cada producto puede consultarse individualmente mediante */api/products/:id*.
- **Producto: Source (N : 1 lógica):** Cada producto incluye un campo *source* que indica su origen. No existe una tabla aparte para las fuentes(source), pero varios productos pueden compartir el mismo valor.

5. Decisiones de dataset que afectan a la app

- Se conservan *id*, *title*, *price*, *currency*, *image*, *url* y *source* porque son los campos usados por listado, detalle y filtros.
- Se descartan descripciones largas y metadatos no usados para reducir tráfico y complejidad en Android.
- **birthDate** se captura en la interfaz como **dd/MM/aaaa** y antes de enviar se transforma a **yyyy-MM-dd**, que es el formato esperado por la API.
- Los identificadores (id) se conservan como numéricos porque el backend los devuelve como enteros y eso facilita la navegación y la lectura de detalle.